

Breaking Google Wallet For Fun and Profit

Etienne Nedjai^[0009-0001-5398-7203] and Mohamed Sabt^[0000-0002-9540-5363]

Univ Rennes, CNRS, IRISA
{etienne.nedjai,mohamed.sabt}@irisa.fr

Abstract. Digital wallets on smart devices, such as Google Wallet, enable the storage of a wide range of credentials, including payment cards, transit passes, and identity cards. Despite their widespread adoption, their security guarantees have received limited research. In this paper, we analyze the security architecture of Google Wallet, which served more than 200 million users worldwide in 2025. We consider a threat model in which a user with elevated privileges on their own device seeks to manipulate stored credentials for fraudulent purposes. Our analysis focuses on the Device Encryption Key (DEK), which protects the Wallet’s sensitive data. Here, we investigated the DEK protection during its whole lifecycle, identifying several weaknesses that lead to DEK recovery. Using the recovered DEK, we decrypted all the Wallet assets in order to characterize the nature of the protected contents that are stored on the device. Finally, we present the “hack-and-go” attack, which allows a user to arbitrarily increase the balance of a stored transit card, demonstrating the practical impact of our findings.

Keywords: Google Wallet · Reverse Engineering · Software Analysis.

1 Introduction

Context and Motivations. Digital wallets have experienced remarkable deployment growth over the past decade, evolving far beyond their original purpose as simple payment tools. Initially designed for online and point-of-sale transactions, they rapidly expanded to support, among others, vaccination attestation, loyalty card aggregation, event ticketing, and transit passes, effectively consolidating functions previously distributed across multiple stakeholders into a single mobile app [39]. This wide range of use cases is reflected in adoption figures of considerable scale: as of 2025, approximately 4.5 billion individuals used a digital wallet, with projections indicating this figure will surpass 6 billion by 2030 [38]. Digital wallets are therefore no longer a niche financial app, but an increasingly central component of everyday digital life.

Here comes Google Wallet as a major player in this field with more than one billion downloads and 250 million active users [21]. This is due, as shown in [15], to superior usability and tight integration with other Google services. Moreover, Google leverages its dominant brand in the Android ecosystem to attract more users to their Wallet¹. As such, users trust that Google Wallet would protect

¹ *It’s not a wallet. It’s Google Wallet.* [51]

their sensitive data better than any other Android wallet app, by leveraging, as self-claimed, *advanced security* mechanisms [50].

Unfortunately, despite its wide adoption, few works have provided insights into the overall security of Google Wallet. In particular, we had to use a web archive to retrieve the most relevant document provided by Google describing the overall security design of Google Wallet (née as Google Pay) [26]. In this document, it is underlined that the Wallet encrypts its sensitive data using a key coined the “Device Encryption Key” (DEK) that is loaded into the secure hardware of the Android device, such as the Trusted Execution Environment (TEE) [5]. Thus, some might reasonably expect that the Wallet data cannot be extracted even by root attackers, thereby providing strong protection for, among others, users’ payment and transit cards entrusted to Google Wallet.

Contributions. The exact role of the aforementioned DEK remains unknown. Of particular interest, the Wallet security paper mentions two concerning design choices [26] : (1) a copy of the DEK is kept on the device’s memory, and (2) the DEK is stored on a *PendingIntent* [19] that survives until reboot. Such design casts serious doubts about the effective threat model Google Wallet defends against, thereby raising questions about compliance in sectors where effective TEE protection is mandated, such as transit cards and digital ID cards [24].

Our work aims to push this topic forward, as the protected assets are high-stakes. This paper fills the gap by providing the first thorough security analysis of Google Wallet to independently assess its design beyond what was roughly specified in [26], which was last updated before its archive in April 2023. Being proprietary and closed-source, our work required complete reverse engineering of the underlying security mechanisms, particularly every component related to the DEK. To achieve this, we apply a two-pronged approach combining static analysis via decompilation with dynamic memory inspection.

Accordingly, this work presents four main contributions. **First**, we depict the internal workings of the Wallet Android service, highlighting how its components interact to protect sensitive data. **Second**, we investigate the DEK protection through its whole lifetime. Here, we look to confirm or contradict what was outlined in [26], namely, an unprotected copy of the DEK is kept in a *PendingIntent* that persists beyond the Wallet process lifetime, which gives (root) attackers a universal entryptpoint to retrieve it. In this regard, we establish that Google Wallet does not protect the confidentiality nor the integrity of its sensitive data against root attackers despite its reliance on TEE. **Third**, we decrypt the whole protected database of Google Wallet, and find insights on the actual stored assets for various cards. **Finally**, based on our findings, we define a new attack, *hack-and-go*, enabling an attacker to add any amount of money to their transit cards. The attack consists of three steps: (1) the DEK is retrieved from a rooted device and used to decrypt the protected database of Google Wallet, (2) the different secret keys of the installed transit card are identified and recovered, (3) these keys are later used to act as a legitimate transit reader that can modify the balance of the transit card through some standard NFC commands without

having to manually alter the Wallet filesystem. Optionally, the attacker can later move its card to another device that might not even be rooted, and the same forged NFC commands would still succeed.

To sum up, our investigation shows that Google Wallet does not perform better than other wallet apps in terms of security. Indeed, it just relies on the standard security mechanisms provided by Android for all apps, which can be bypassed by a powerful attacker [34]. Hopefully, our findings would help the different stakeholders to better understand the effective threat model Google Wallet defends against before entrusting it with their confidential data.

Threat Model. We adopt a *root attacker model* where the attacker has full control over their own device. This level of access can be achieved, for example, by *rooting* their device. Typically, a root attacker is often disregarded from the Android threat model for most apps [34]. However, given the purpose of Google Wallet to be a universal app for all our daily cards [50], it seems necessary to take the root attacker model into account, especially when the attacker would greatly benefit from compromising their own Google Wallet. In this paper, we show how an attacker can add an arbitrary amount of money to their transit cards installed on Google Wallet, which creates a real financial incentive motivating such a powerful threat model. More broadly, we show that Google Wallet is not suitable for any application in which a Man-at-the-End (MATE) attacker is applicable [2], thus discarding many use-cases, including transit cards and government-issued ID cards. A detailed description of the attacker model we consider is presented in Section 3.1.

Open Science. In support of open science, we release our artifact on GitHub [8], which includes: (1) a script to retrieve Myki card authentication keys, and (2) an app to perform credit operations on the target Myki card.

2 Background

2.1 Android Security Model

Android enforces a dual-world isolation model grounded in ARM TrustZone [12], partitioning the device into two isolated execution environments [42]: the Normal World and the Secure World.

Normal World. The Android OS, apps, and system services operate in the Normal World. While Android employs multiple defense layers here (Linux kernel isolation, sandboxed app processes, permission enforcement, and SELinux policies), the Android security model, as described in [34], explicitly considers that the Normal World remains inherently vulnerable against some potential OS-level compromise, especially when the attacker acquires the *root* privilege.

Secure World. The Trusted Execution Environment (TEE) is an isolated component that runs a dedicated Trusted OS [43], such as Google Trusty [6] or vendor-specific solutions (*e.g.*, Qualcomm QSEE [36], Samsung TEEgrs [45]).

The TEE guarantees code integrity and data confidentiality even in the event of a full kernel compromise, effectively defending against root.

2.2 Android Keystore

The Android Keystore API [5] provides cryptographic services for any Android app through the standard Java Cryptography Architecture. Since Android 7 [4], devices must have a TEE-backed Keystore by default to pass Compatibility Test Suites (CTS). The Keystore is central to Android security (*e.g.*, phone unlocking) and widely used by sensitive applications, with 44% of such apps relying on it [13]. Its popularity stems from its strong protection against powerful attackers, including root, by confining key usage to the TEE.

2.3 Google Wallet

Google Wallet, initially launched in 2011 as a mobile payment service, merged with Android Pay in 2018 to form Google Pay (GPay). The original Wallet app became Google Pay Send, a peer-to-peer payment service, before being discontinued in 2020. GPay allowed users to make in-store purchases, peer-to-peer transfers, and online payments using linked cards. In 2022, GPay was rebranded as Google Wallet to also include items like loyalty cards, IDs, transit passes, and event tickets. Like GPay, the new Wallet supports contactless payments via *tap-and-pay*, available in over 100 countries. However, certain functionalities, like specific transit cards, are region-dependent.

Internally, Google Wallet heavily depends on the Google Mobile Services (GMS) to implement its security features. GMS is a set of proprietary, closed-source apps and APIs developed by Google and preloaded on numerous Android devices [3]. GMS consists of three main components: the Google Play Store, Google Play Services, and Google apps, such as Google Maps, Google Chrome, Gmail, and YouTube. In this paper, we are only interested in Google Play Services, as they offer important security features for Google Wallet.

3 Problem Statement

3.1 Threat Model

Despite their mass adoption, the threat model of mobile digital wallets has never been properly defined; namely, it is not well established what “level of protection” a wallet app should achieve against a powerful attacker. Developers of these apps tend to reassure their users about the security of their solution, while never providing an independent assessment of their leveraged security architecture. Paradoxically, at the same time, they minimize any security breach by insisting that the Android Security Reward Program mostly discards any reported issue requiring root for exploitation [25]. As a matter of fact, the scope of rooted devices is quite limited: they only represent 0.1%-0.3% of the ecosystem [33, 49].

In other words, developers consider that attackers are not going to undertake the pain of rooting their device for too little benefit, thereby excluding such an attack vector.

In this work, we argue the importance of Google Wallet to include rooted devices in its threat model. Underlined reasons are twofold. First, Google Wallet, or more precisely, the GMS wallet service, is installed on almost all Android-certified devices. Its wide adoption makes it an enticing target for attackers, especially since it accepts cards from numerous stakeholders worldwide, including banks and governments. One compromise may lead to global fraud, especially since it locally stores important assets: payment cards, prepaid transit cards, and national ID cards. Thus, it is reasonable to assume that users would actually desire to break into their own Wallet for profit, which brings us close to the Man-At-The-End (MATE) model [2], in which an attacker is presumed to have unrestricted capabilities on their device.

In this paper, we do not consider all the capabilities of a MATE (aka root) adversary. Indeed, we minimize the actions requiring root in our final exploitation; we only needed access to the wallet filesystem, as well as sending an intent within the GMS process. In this regard, exploitation may become more challenging, but any identified weakness would become more portable across different GMS versions, hence technically more applicable to long-term real-world scenarios. As such, we exclude software-based attacks that exploit software vulnerabilities in the Wallet app, since such vulnerabilities would depend on the GMS version and can be easily patched. Moreover, we assume the cryptographic implementations of the Wallet app are secure, including the Android Keystore.

3.2 Main Goal

The starting point of our study is the aforementioned claim in [26]: the DEK is both loaded in the TEE and kept in the device’s memory. This claim is quite peculiar with two contradicting threat models, since it implies that the TEE is leveraged to protect against root attackers that can still recover the DEK because an unprotected copy is kept in memory. In this paper, we first investigate whether this claim still holds in the latest version of Google Wallet. Then, we explore how easy an attacker can exploit such a bad design in a fully updated Android environment. A weakness does not necessarily imply real-world exploitation. For example, Google Wallet might have implemented sophisticated protection mechanisms that make any attempt to obtain the DEK time-consuming even for root attackers. Our purpose is to thoroughly uncover the weaknesses in the security mechanisms related to DEK protection.

Overall, we investigate the security architecture of Google Wallet, while focusing on the Device Encryption Key (DEK) in order to address the following question: against a root attacker, to what extent is it possible to recover the key value of the DEK in order to fully break the confidentiality and the integrity of all the Wallet data. By addressing this question, we show that Google Wallet does not provide more besides the standard security mechanisms offered by An-

droid to all apps. Accordingly, in Section 4, we present our empirical study that answers the following questions:

- **RQ1.** Where is the DEK in Memory?
- **RQ2.** What is the DEK Lifetime?
- **RQ3.** What do the Wallet Assets Store?

4 Research Questions

4.1 Experimental Settings

We use various tools to answer our research questions: jadx [48] v1.5.2 for APK decompilation, frida [37] v17.2.12² for function hooking and dynamic memory inspection, and an OmniKey AG CardMan 5022 NFC reader [28] to interact with payment, loyalty, and transit cards in a controlled environment. Moreover, for our experiments, we use two smartphones running two different Android versions, namely a Pixel 8a with Android 16 and a Nokia G20 with Android 12, to ensure that our observations are not limited to a specific Android version. In our study, we find no difference between these two devices regarding the security mechanisms that we analyze in Google Wallet. Device specifications and GMS versions are listed in Table 1.

Table 1. Environment for our experimentations

	Pixel 8a	Nokia G20
Wallet APK version	26.21.918907778	25.15.749847794
GPlay Services version	25.20.39(260400-775404632)	25.16.33(190400-750736914)
Android version	16	12
Build	BP3A.251005.004.B2	00WW_2_360_SP01

4.2 RQ1: Where is the DEK in Memory?

The Device Encryption Key (DEK), as defined in [26], is the key that is used to *encrypt and decrypt the sensitive data on the device*. The same document also states that the DEK is *kept in the device’s memory* when required for cryptographic operations. Our goal is therefore to pinpoint the DEK location in process memory during execution, and if found, to examine its runtime protections.

Methodology. Recall that Google Wallet heavily relies on GMS (or more precisely, the GMS wallet service) to manage its sensitive data. Thus, henceforth, our focus will be on the GMS process rather than the Wallet app. GMS consists

² and frida-tools v14.4.4

of numerous components that interact with each other, and therefore, a naive reverse engineering would be time-consuming to find any reference to the DEK.

Instead of blindly analyzing such a huge base of code (161 MB), we start from the assumption that if the TEE is involved in the DEK protection (refer to [26]), GMS would necessarily go through the Android Keystore API. We therefore list all Keystore keys associated with the GMS wallet service by accessing the Keystore database and enumerating keys linked to the GMS UID. Since wallet is just a single GMS service, not all keys are wallet-related. To refine the candidates, we control the Keystore process to intercept key-access calls. Prior to execution, we remove all cards and clear GMS storage. We then interact with the Wallet features, including adding and using cards, while monitoring Keystore usage. To answer RQ1, we proceed in two steps. First, we look for all references of the `setEntry` Keystore API, since we assume the DEK is provisioned from some remote server (refer to [26]), before being imported into the TEE. If no reference is found, we suggest that the DEK is not directly stored in the TEE, but instead resides in GMS persistent storage, possibly protected by a TEE-backed key. Thus, secondly, we analyze all Keystore usage in order to find the DEK.

Observations. We begin by listing all the Keystore keys that we identified as related to the GMS wallet service. Following the first direction, we observe two calls to `setEntry` in GMS; however, none of the corresponding aliases appear in the Keystore database. As a consequence, we go for the second direction even though it contradicts [26]. Here, we analyze each Keystore key used by the wallet service and examine its purpose. We identify two relevant keys, *pay_storage_key* and *storage_key_alias_2*. After a reboot, we observed that these keys are used to decrypt some data while respectively instantiating *StorageKeyBundle* and *StorageKey* objects within GMS. Notably, from an empty Wallet, the first key is created when Google Wallet is launched, while the second is generated when a card is first added. In addition, we notice that deleting these keys triggers a `GET_STORAGE_KEY` remote request via a TLS connection before processing any sensitive data.

To sum up, the key in *StorageKey* object presents the following characteristics: (1) it is provisioned from some Google servers, (2) it is encrypted using some Keystore key, and (3) its value does not change when deleted, as we never noticed any data re-encryption. All these hint to the DEK. To confirm our assumption, we pursue our investigation by performing automatic dependency analysis regarding the *StorageKey* class. We find that the key of *StorageKey* is used to encrypt some data when adding a new card, and to decrypt that data when using the same card. We therefore conclude that it must be the DEK, as it closely fits the definition provided in [26].

Having identified the DEK, we apply dynamic instrumentation and memory inspection to evaluate whether the DEK can be recovered from process memory at runtime. Using Frida instrumentation on the main GMS process, we intercepted the initialization of a *StorageKeyBundle* instance and observed that the DEK is loaded into memory in cleartext, without apparent runtime protection.

We further identified a *StorageKey* instance within the persistent GMS process. Similar to *StorageKeyBundle*, the DEK stored in this object is present in cleartext in process memory. Our analysis shows that both objects contain the same DEK value, indicating that it is replicated across multiple runtime contexts.

Insights. Our analysis shows that the DEK is present in two distinct runtime objects, *StorageKeyBundle* and *StorageKey*, within the GMS process memory. In both cases, the key is stored in cleartext and remains accessible during execution, making it recoverable by an adversary with the ability to inspect process memory. The next question concerns the window size of this opportunity.

The DEK is never loaded into the TEE, either persistently or at runtime. Instead, it is stored encrypted by a Keystore key. Importantly, the DEK is loaded into the runtime memory of the GMS wallet service and remains unprotected while in use.

4.3 RQ2: What is the DEK Lifetime?

If considering our threat model as introduced in Section 3.1, a root attacker can, in theory, recover any key whenever an unprotected copy is loaded into a process memory. However, the actual exploitation differs greatly depending on the key lifetime. Indeed, it is more difficult to recover a key that is wiped off memory directly after its usage, compared to a key that remains loaded indefinitely. In this question, we analyze the DEK lifetime once it has already been provisioned and persistently stored encrypted.

Methodology. We add some cards to our Wallet and use them. Then, we trace the DEK lifetime after two different events: stopping the Wallet app and restarting the GMS process. To this end, we leverage dynamic instrumentation to hook the different methods of the *StorageKey* and *StorageKeyBundle* classes for memory inspection. We also hook the corresponding constructors to analyze the associated thread stack traces to identify the origin of the DEK values used during object initialization. Our purpose is to assess whether the DEK is securely erased from memory after one of these events has occurred.

Observations. Concerning the first event, we run multiple instances of Google Wallet and repeatedly access the DEK in *StorageKey* until stopping the Wallet app. We find that, while resident in memory, the DEK is never cleared and can be recovered by locating a *StorageKey* object in the process memory. We also notice that the GMS wallet service does not react to Wallet being stopped, and keeps the DEK in its memory without any protection.

As for the second event, *i.e.*, the wallet service being restarted, we started by monitoring the Keystore, since we expected that the wallet would seek its persistent DEK that is encrypted using the Keystore. However, no call to the Keystore was observed, even though the DEK was properly initialized into the wallet memory. This implies that there is another copy of the DEK somewhere

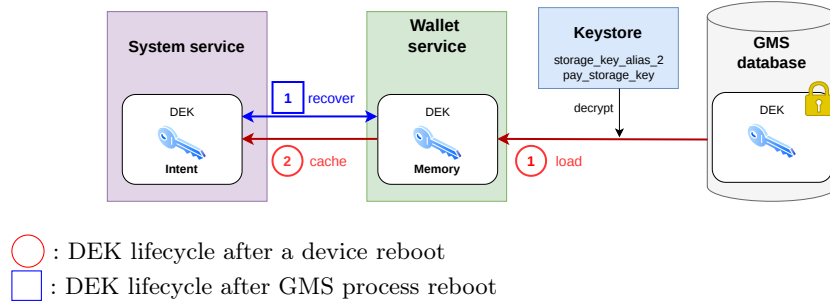


Fig. 1. The DEK lifecycle

in the device memory, possibly in a *PendingIntent* as specified in [26]. A *PendingIntent* [7] is an Android system token that encapsulates a future operation and remains valid even if the creating process is terminated.

Thus, we closely inspect how the different memory objects are initialized during a device reboot by tracing the DEK decryption process as our starting point. We summarize our findings as depicted in Figure 1: the decrypted DEK is first loaded into memory and subsequently encapsulated within an *Intent*. This intent is then sent to the *StorageKey cache service* via a *PendingIntent*. Based on this, we monitored *Intent* calls after restarting the GMS process, and observed that the DEK is actually recovered through the *PendingIntent* associated with the *StorageKey cache service*. We think that the rationale behind such a design is to improve the required time to recover from the GMS process being abruptly restarted.

Insights. The DEK remains in GMS process memory for a long time, contrary to best practices recommending sensitive data be scrubbed when unused [16]. To ensure availability across process restarts, an additional copy of the DEK is stored within a *PendingIntent*. Our analysis shows that this copy is also maintained in cleartext and lacks additional runtime protection. As a result, an attacker capable of interacting with the *StorageKey cache service* could retrieve the cached DEK by querying the associated *PendingIntent*. In practice, however, access to this service is restricted to GMS components, which restricts exploitation to attackers being able to control a process with the same UID as GMS.

The DEK is never wiped from process memory after being loaded. A clear-text copy is cached in a *PendingIntent* that survives process restarts.

4.4 RQ3: What do the Wallet Assets Store?

The findings from RQ2 suggest that a root attacker has ample opportunity to recover their own DEK, and therefore decrypt the sensitive data stored in the Wallet. This research question examines the nature of such data and evaluates

its potential value for an attacker. Our purpose in RQ3 is to focus on studying data semantics; real-world full exploitation will be deferred to Section 5.

Methodology. Google Wallet supports a multitude of cards, and each of them introduces a plethora of proprietary assets. We prioritized lucidity in our presentation and longitudinal analysis over completeness. To ease understanding, we only evaluate two cards: a Visa payment card and a loyalty card. Transit cards are studied in Section 5. We selected these two cards because they nicely picture the working of Wallet: a Visa card can be used offline, while a loyalty (and event) card mainly works online. As such, we expect that their assets would represent what other Wallet cards include.

Observations. To answer RQ3, we needed to reverse-engineer the cryptographic operations involved in data decryption. We find that the DEK is first processed through an HMAC-based key derivation function (KDF) to generate encryption and MAC keys. Further static analysis reveals an Encrypt-then-MAC construction combining AES-CBC for confidentiality and HMAC-SHA256 for integrity. Encrypted data is stored in the *pay* and *android_pay* databases, which also contain the Keystore-encrypted DEK.

Regarding the Visa payment card, we find the corresponding tokenized card in the encrypted database. As a matter of fact, Google Wallet implements the EMV Payment Tokenization standard for contactless payments, replacing the physical card with limited-use payment tokens [22]. A valid transaction relies on four key elements: the virtual account number (linked to the underlying payment account), the limited-use key (LUK) used to generate transaction cryptograms, the Integrated Circuit Card (ICC) key pair used to authenticate card data to the terminal, and the Application Transaction Counter (ATC), which is incremented for each transaction to mitigate replay and cloning attacks. Our analysis of the decrypted payment card revealed all four elements. Thus, a valid payment transaction can be built by any attacker in possession of these elements, even outside the provisioned device. To confirm this, we cloned the tokenized card onto another device by re-encrypting data with the appropriate DEK and manually injecting it into the Wallet database. Using the copied card, we successfully performed a transaction on a commercial payment terminal, completing our cloning attack.

As for loyalty cards, they mostly store the displayed data on the Wallet interface (*e.g.*, card name). Of particular interest, they also store the card’s account identifier that enables the issuer’s backend to identify the corresponding user account and retrieve associated account data. Despite their low value, we could not decrypt loyalty cards using the recovered DEK. Through some static analysis, we find that these cards, by default, are AES-GCM encrypted using a different key that we did not identify previously. However, we also find that Wallet defines a fallback mechanism to use the DEK in case the decryption with that key fails. Thus, we trigger a decryption failure, allowing us to easily access the protected account identifier with the DEK.

Insights. Wallet assets fall into two categories: (1) card-centric, where sensitive data is stored directly on the card, and (2) account-based, where sensitive information is maintained on the issuer’s backend server, while the card itself only contains identifiers or references used to retrieve the associated account data. Card-centric assets can be copied and used outside the Wallet app, while account-based assets can be overwritten to impersonate other users.

Wallet assets fall into two categories: card-centric and account-based. Both can be recovered and exploited via the DEK.

5 Case Study: The Hack-and-Go Attack

We show that acquiring the DEK provides unrestricted access to all cards stored in the Wallet. Some might question the practical interest of such access, since a root-level attacker is required to obtain the DEK, as presented in Section 4. To cast away any doubt, we will demonstrate how compromising Google Wallet could create great financial benefit for the attacker, which may motivate malicious users to break into their own wallets. In practice, we implement “*hack-and-go*”, in which an attacker can arbitrarily modify the stored monetary balance of their transit card. An important highlight of our attack is that it can be subsequently carried out on unrooted devices after the first compromise on a rooted one. We successfully conducted our attack on a real transit card and freely topped it up with any amount of money we wanted. Below, we first introduce how the Wallet stores transit cards before presenting our attack.

5.1 Transit Cards in the Wallet

The Wallet supports the emulation of transport cards implementing widely deployed protocols, including Calypso, MIFARE DESFire, and ITSO. In our investigation, if we do not consider payment-like cards, we find that all the transit cards supported by Wallet (*e.g.*, Breeze Card, PRESTO, Clipper, and Myki) communicate with gate readers using the MIFARE DESFire protocol. DESFire cards organize data into files protected by fine-grained access rights specifying read, write, and permission-management operations. Access to a given operation requires successful authentication based on the corresponding cryptographic key through a mutual three-pass challenge-response protocol [46]. During this process, the reader and the card prove possession of the same secret key and derive a shared session key used to secure subsequent communications. Among the different file types, *Value Files* are of particular interest, as they are typically used to store the card’s balance value and provide *Debit* and *Credit* operations for readers possessing the appropriate read and write permissions.

After examining the transit pass data within the Wallet databases, we observed that card files were accessible only in read-only mode, while authentication keys remained encrypted under the DEK. The data describing the card is

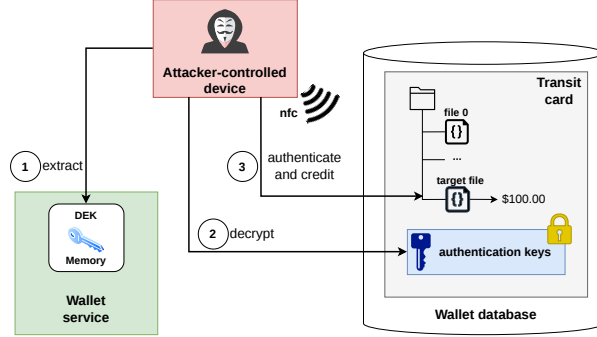


Fig. 2. Overview of the Hack-and-Go Attack

stored in JSON format, including DESFire-specific metadata such as protocol version (version 1 for all observed cards), file metadata (name, content, permissions, and type), and key metadata (key identifiers and associated cryptographic algorithms, *i.e.*, AES or DES). When decrypting the authentication key files with the DEK, a protobuf structure is obtained containing the key identifiers and their corresponding values.

5.2 Technical Implementation

To implement our attack, we consider a card that uses a *Value File* to maintain the stored balance and supports *Credit* operations for balance updates. Figure 2 illustrates how an attacker possessing the DEK can emulate a legitimate top-up operation and arbitrarily increase the card balance. In step ①, the attacker retrieves the DEK either by inspecting the Wallet’s memory or by interacting with the *PendingIntent* exposed by the *StorageKey cache service*. This step is done only once, since we showed that the DEK remains unchanged unless a factory reset is performed. In step ②, the attacker uses the DEK to decrypt the card’s authentication keys and stores them locally. Finally, in step ③, the attacker authenticates to the target *Value File* using the corresponding permission key via NFC communication. Once authenticated, the attacker can issue a *Credit* operation with an arbitrary amount and commit the transaction, increasing the card balance up to the file’s maximum value.

Note that the scope of the attack is extended beyond the attacker’s controlled environment if the card’s authentication keys remain unchanged after card transfer. Indeed, this allows the attacker to continue interacting with the card on other devices without requiring access to their DEK. In this regard, we define the following scenario: malicious users with legitimate devices can ask an attacker possessing a rooted device to recover the authentication keys of their cards. Here, users can simply buy an empty card, ask the Wallet to move it to the attacker’s device, and then get it back along with the required keys. The at-

tacker may charge users for this operation. Finally, users embed these keys into any compliant implementation that can issue the needed *Credit* operation for their card. The user’s Wallet would process these commands as they come from a legitimate top-up, updating the balance file accordingly. Note that all parties involved in this scenario reap benefits from compromising Google Wallet. More importantly, we can see that one single rooted device can be leveraged by many other malicious users.

5.3 Evaluation

We successfully conducted our attack on Myki, an Australian transit card that stores its balance directly on the card. We use the same experimentation settings as specified in Section 4.1. In addition, we use a Pixel 9a as the NFC-enabled device to send the *Credit* NFC commands to the compromised card. To this end, we developed an Android app in Kotlin to issue APDU-wrapped MIFARE DESFire v1 commands via the IsoDep protocol stack. The attack procedure first issues a *Select* command to target the relevant file, followed by *Authenticate* using the key that grants read and write access to the *Value File*. We then apply the *Credit* command to modify the stored balance by an arbitrary amount, and finalize the transaction using *CommitTransaction*. Interestingly, after transferring the card to a different device, the modified balance persists. This suggests that the backend system does not invalidate or reconcile unauthorized balance changes during the transfer process. Furthermore, we observed that the app can still issue *Credit* commands on the new device, which indicates that, in our experimental setting, the authentication keys remain unchanged after transfer.

5.4 Variants of Hack-and-Go

While Myki stores its balance on the card, other transit systems may instead maintain account state on some backend server, such as Breeze Card, Clipper, and PRESTO. Although this would hinder direct balance modification, the attacker can still leverage its knowledge of the card’s authentication keys differently. Indeed, in addition to the balance file, a card may also contain a proof of transport subscription even if the transit card is account-centric. In this case, an attacker possessing a valid annual transit pass, for instance, could share it with other users, potentially allowing them to access the service without purchasing their own subscription. The same techniques are applied to use the authentication keys to read or overwrite the content of the appropriate card files.

6 Discussion

6.1 On the Integrity of Wallet Data

Throughout this paper, we mainly focused on the impact of DEK recovery on the confidentiality of Wallet sensitive data. Regarding integrity, one might expect that modified data re-encrypted with the correct DEK would be accepted.

In our experiments, we notice that this does not work as expected. Here are our observations: the Wallet accepts processing our modified data (*e.g.*, adding new cards) as long as the device running the compromised Wallet remains offline. Once the device is connected to the Internet, the Wallet automatically synchronizes with Google servers, overwriting any changes made on the device to match the data stored in the cloud. This process thus removes our modifications, since they were not officially added through the attacker’s Google Account. We attempted to block synchronization by making the GMS databases read-only, but GMS requires write access at runtime. As a result, integrity is not impacted by DEK recovery if the attacker needs to stay online, which limits its impact.

6.2 Mitigation

Our results suggest that the main cause of our findings is architectural: a root attacker can recover the DEK as long as a copy is kept in the device’s memory. To fix, ideally, the Wallet boundary processing any sensitive data should be suppressed from the Normal World, so that no secret appears in memory. Thus, an effective mitigation is to implement the GMS wallet service as a TEE application, so that a root attacker cannot extract keys from its memory. Since we do not expect Android to abandon the modular architecture created by its GMS service, nor to sacrifice efficiency, we propose some other mitigations intended to make our attack more difficult to exploit in practice by reducing DEK exposure. Accordingly, we suggest the following patch: loading the DEK into the TEE, so that any related computation must go through the Android Keystore API. This implies suppressing the DEK copies that exist in the *StorageKey* object and the *PendingIntent*. Now, we compute the resulting overhead of using the TEE-based Keystore in the context of Google Wallet.

In the environment described in Table 1, we evaluated the overhead of decrypting a 1536-byte (*resp.* 220-byte) ciphertext, corresponding to the payment card secret values (*resp.* transit card authentication keys) processed by the DEK, using AES-CBC. When using a key stored in the process memory, we achieved an average decryption time of 0.4 ms (*resp.* 0.4 ms). In contrast, retrieving a key stored in a service via intents resulted in an average time of 6 ms (*resp.* 6 ms). Compared to the current Wallet implementation, directly using the Keystore expectedly increased the average execution time to 18 ms (*resp.* 14 ms). Although this represents a 45-fold (*resp.* 35-fold) increase in overhead compared to using a key in process memory, we still recommend using the Keystore for all DEK-related operations, as the resulting overhead remains well below the 500 ms (*resp.* 500 ms) maximum tap latency requirement for EMV transactions [23] (*resp.* transport applications [1]).

6.3 Responsible Disclosure

To prevent any fraud, all experiments were conducted on local resources fully under our control. No money was stolen, since when evaluating the attack in practice, we did not defraud the transportation company by claiming to refund

the money that we added onto our transit card. Following responsible disclosure, we reported our findings to Google via the Android Security Reward Program with a 90-day window. Google acknowledged the report but disputed its relevance, expectedly concluding after 78 days that it was ineligible because “*the described attack requires root access to the victim’s phone*”. Google agreed to publication, although the issue was closed as “*Won’t fix (intended behavior)*”, meaning our provided scripts may still work if conditions are met. In essence, Google’s Bug Hunters Team conveyed that the issue might be fixed despite being considered low priority. Two months later, we found that Android pushed an update enforcing the rules related to root detection while using the Wallet. However, they kept all the identified weaknesses in the Wallet architecture.

This tension returns us to the core threat model: what should mobile digital wallet apps, and Google Wallet in particular, protect against? While our reasoning is detailed in Section 3.1, we argue that Google Wallet should address our attack for two additional reasons. First, it is unconventional to protect the same key using TEE while simultaneously exposing it through a long-lived *Pending Intent*. As such, we deem it important for such an app handling highly sensitive data not to solely rely on a simple control rule related to intent access or root detection. Second, Google Wallet steadily grows its protected assets to attract new users. For example, Google Wallet has recently announced that it will support digital IDs for age verification in select EU states. In this sensitive context, our results timely show that Wallet does not meet European age-verification requirements that mandate strong security guarantees anchored on hardware (*e.g.*, TrustZone or Strongbox-based TEE).

Overall, the current state of affairs is that, once rooted, an attacker can effortlessly compromise all sensitive data in Google Wallet. Although acquiring root privilege might be challenging, we believe that users might willingly downgrade the security of their device for their own benefit.

7 Related Work

7.1 Man-at-the-End Attacks on Android

Our work approach follows the *Man-at-the-End* model, in which the adversary has full control over their own Android device, including root privileges, and attempts to recover or misuse sensitive information provisioned by a remote issuer that considers the device a source of trust. Similar attacker models have been studied in the context of Digital Rights Management (DRM) systems [35], of the Android Protected Confirmation (APC) protocol [11], and of the recovery of cloud services authentication credentials [53]. To protect issuer-provisioned secrets against privileged attackers, modern Android apps rely on hardware-backed security mechanisms, such as the TEE. Despite their strong guarantees, several works have demonstrated vulnerabilities in TEE implementations, including Samsung TEEgris [47], Qualcomm QSEE [29], and Huawei TrustCore [14]. However, they still provide better security compared to the Wallet design that stores its secret key in a *PendingIntent*. Finally, prior work has also identified

weaknesses in the Android Keystore and Keymaster infrastructure [31,44], which are widely used to protect app cryptographic keys. Unlike previous work that focuses on compromising the Keystore or the TEE itself, this paper demonstrates how a privileged attacker can universally recover issuer-provisioned secrets from Google Wallet.

7.2 Digital Wallets

Alongside Google Wallet, which is estimated to have over 250 million active users [21], other major players exist: Apple Pay (approximately 744 million users) and Samsung Wallet (approximately 150 million users) [20]. These apps are the leaders in the global digital wallet ecosystem, even though their dominant position is challenged in some countries by some regional apps, such as WeChat Pay in China and Osaifu-Keitai in Japan. Prior work has mainly focused on the security of payment functionalities within digital wallets. Overall, three main classes of attacks have been identified against contactless payment systems: relay attacks [32,41], eavesdropping attacks [17,27], and skimming attacks [40]. These studies primarily address NFC-based payment card interactions and operate under a threat model that centers on financial credentials. Additional studies have highlighted trust issues between banks and digital wallets [10,18]. However, these studies do not consider the broader set of assets managed by modern digital wallets, which increasingly extend beyond payment cards. Our work instead considers the core security mechanisms of the Wallet itself, which makes our work apply to all the Wallet assets. The recent integration of identity credentials into mobile wallets further expands the wallet attack surface; however, only limited prior work has analyzed their security on Android devices [9,30,52]. In this paper, due to the limited geographic availability of the Wallet identity features (notably restricted to the United States and the United Kingdom at the time of writing), we were unable to empirically evaluate the assets stored by Google Wallet related to this feature.

8 Conclusion

In this paper, we analyzed the security mechanisms of Google Wallet to protect its assets. Through reverse engineering of the wallet service, we thoroughly studied the lifetime of the cryptographic key that is identified to encrypt all sensitive data. Our analysis revealed a critical weakness related to this key management, as it remains in plaintext within the Wallet process memory and is never erased. From this weakness, we demonstrated the “*hack-and-go*” attack, which allows an attacker to arbitrarily increase the balance of their transit cards installed on Google Wallet. As future work, it would be interesting to investigate the security protocol defining the card provisioning process as implemented by Wallet. In addition, extending the analysis to digital identity documents could provide valuable insights as their adoption continues to grow.

References

1. Agent, U.T.: Contactless latencies. <https://unitiag.com/contactless-latencies/>
2. Akhunzada, A., Sookhak, M., Anuar, N.B., Gani, A., Ahmed, E., Shiraz, M., Furnell, S., Hayat, A., Khan, M.K.: Man-at-the-end attacks: Analysis, taxonomy, human aspects, motivation and future directions. *J. Netw. Comput. Appl.* **48**, 44–57 (2015)
3. Android: Google mobile services. <https://www.android.com/gms/>
4. Android: Android 7.0, (n) compatibility definition. https://source.android.com/docs/compatibility/7.0/android-7.0-cdd#9_11_keys_and_credentials (2016)
5. Android: Android keystore system. <https://developer.android.com/privacy-and-security/keystore> (2025)
6. Android: Trusty tee. <https://source.android.com/docs/security/features/trusty> (2025)
7. Android: Pendingintent. <https://developer.android.com/reference/android/app/PendingIntent> (2026)
8. Anonymous: Hack-and-go artifacts. <https://anonymous.4open.science/r/fdsqfsdfsL-C5C5> (2026)
9. Ansaroudi, Z.E., Sciarretta, G., Maria, A.D., Ranise, S.: Navigating secure storage requirements for EUDI wallets: a review paper. *EURASIP J. Inf. Secur.* **2025**(1), 2 (2025)
10. Anwar, R.H., Hussain, S.R., Raza, M.T.: In wallet we trust: Bypassing the digital wallets payment security for free shopping. In: *USENIX Security Symposium*. USENIX Association (2024)
11. Arapinis, M., Danos, V., Racouchot, M., Robin, D.A.R., Zacharias, T.: Attacking and fixing the android protected confirmation protocol. In: *EuroS&P*. pp. 458–479. IEEE (2025)
12. ARM: Arm trustzone technology. <https://developer.arm.com/documentation/100690/0201/Arm-TrustZone-technology>
13. Blessing, J., Anderson, R.J., Beresford, A.R.: Keydroid: A large-scale analysis of secure key storage in android apps. *CoRR* **abs/2507.07927** (2025)
14. Busch, M., Westphal, J., Müller, T.: Unearthing the trustedcore: A critical review on huawei’s trusted execution environment. In: *WOOT @ USENIX Security Symposium*. USENIX Association (2020)
15. Chakraborty, D., Siddiqui, A., Siddiqui, M., Rana, N.P., Dash, G.: Mobile payment apps filling value gaps: Integrating consumption values with initial trust and customer involvement. *Journal of Retailing and Consumer Services* **66**, 102946 (2022)
16. Chapman, R.: Sanitizing sensitive data: How to get it right (or at least less wrong...). In: *Ada-Europe. Lecture Notes in Computer Science*, vol. 10300, pp. 37–52. Springer (2017)
17. Choi, D., Lee, Y.: Eavesdropping one-time tokens over magnetic secure transmission in samsung pay. In: *WOOT. USENIX Association* (2016)
18. Chothia, T., Clee, A., Boureanu, I., Pavlides, G.: More is less: Extra features in contactless payments break security. In: *USENIX Security Symposium. USENIX Conference Proceedings*, USENIX Association (2025)
19. Developers, A.: Pendingintent. <https://developer.android.com/reference/android/app/PendingIntent> (2026)
20. Elad, B.: Samsung pay statistics 2026: Massive user growth. <https://coinlaw.io/samsung-pay-statistics/> (2026)

21. Emewulu, T.C.: Apple pay vs google pay: 2025 statistics, adoption rates, and surprising trends. <https://www.chargeflow.io/blog/apple-pay-vs-google-pay-statistics-adoption-rates-market-share> (2025)
22. EMVCo: Emv® payment tokenisation. <https://www.emvco.com/emv-technologies/payment-tokenisation/> (2025)
23. Forum, U.P.: Guidelines for contactless atm transactions - a guide for atm owners and operators. <https://www.uspaymentsforum.org/wp-content/uploads/2019/08/Contactless-ATM-Guidelines-FinalV2-July-2019.pdf> (2019)
24. GlobalPlatform: Mobile id: Realization of mobile identity solutions by globalplatform technologies. Tech. rep., GlobalPlatform (2018), <https://globalplatform.org/resource-publication/mobile-id-realization-of-mobile-identity-solutions-by-globalplatform-technologies/>
25. Google: Security risks with modified (rooted) android versions. <https://support.google.com/accounts/answer/9211246>
26. Google: Google pay security paper. <https://archive.ph/1tH6Q> (2023)
27. Haselsteiner, E., Semiconductors, P.: Security in near field communication (nfc). Workshop on RFID Security (2006)
28. HID: Hid omnikey 5022. <https://www.hidglobal.com/products/omnikey-5022-reader>
29. Khalid, F., Masood, A.: Vulnerability analysis of qualcomm secure execution environment (QSEE). *Comput. Secur.* **116**, 102628 (2022)
30. Kouliaridis, V., Karopoulos, G., Kambourakis, G.: Assessing the security and privacy of android official ID wallet apps. *Inf.* **14**(8), 457 (2023)
31. Lapid, B., Wool, A.: Navigating the samsung trustzone and cache-attacks on the keymaster trustlet. In: ESORICS (1). *Lecture Notes in Computer Science*, vol. 11098, pp. 175–196. Springer (2018)
32. Margraf, M., Lange, S., Otterbein, F.: Security evaluation of apple pay at point-of-sale terminals. In: NGMAST. pp. 115–120. IEEE (2016)
33. Mascellino, A.: Rooted devices 250 times more vulnerable to compromise. <https://www.infosecurity-magazine.com/news/rooted-devices-250x-vulnerable/> (2025)
34. Mayrhofer, R., Stoep, J.V., Brubaker, C., Kravlevich, N.: The android platform security model. *ACM Trans. Priv. Secur.* **24**(3), 19:1–19:35 (2021)
35. Patat, G., Sabt, M., Fouque, P.: Exploring widevine for fun and profit. In: SP (Workshops). pp. 277–288. IEEE (2022)
36. Qualcomm: Overview of qualcomm tee 5.0. https://docs.qualcomm.com/doc/80-88500-4/topic/78_Qualcomm_TEE_5_0.html (2023)
37. Ravnås, O.A.V.: Frida. <https://frida.re/docs/android/> (2025)
38. Research, C.O.S.: Digital wallet statistics. <https://capitaloneshopping.com/research/digital-wallet-statistics/> (2026)
39. Rijo, L.: Google wallet gets eu digital ids, age credential, and direct checkout. <https://ppc.land/google-wallet-gets-eu-digital-ids-age-credential-and-direct-checkout/> (2026)
40. Roland, M., Langer, J.: Cloning credit cards: A combined pre-play and downgrade attack on EMV contactless. In: WOOT. USENIX Association (2013)
41. Roland, M., Langer, J., Scharinger, J.: Applying relay attacks to google wallet. In: 2013 5th International Workshop on Near Field Communication (NFC). pp. 1–6 (2013)
42. Sabt, M., Achemlal, M., Bouabdallah, A.: The dual-execution-environment approach: Analysis and comparative evaluation. In: SEC. IFIP Advances in Information and Communication Technology, vol. 455, pp. 557–570. Springer (2015)

43. Sabt, M., Achemlal, M., Bouabdallah, A.: Trusted execution environment: What it is, and what it is not. In: TrustCom/BigDataSE/ISPA (1). pp. 57–64. IEEE (2015)
44. Sabt, M., Traoré, J.: Breaking into the keystore: A practical forgery attack against android keystore. In: ESORICS (2). Lecture Notes in Computer Science, vol. 9879, pp. 531–548. Springer (2016)
45. Samsung: Samsung teegris. <https://developer.samsung.com/teegris/overview.html> (2026)
46. Semiconductors, N.: Mifare desfire ev1 contactless multi-application ic. https://www.nxp.com/docs/en/data-sheet/MF3ICDX21_41_81_SDS.pdf (2015)
47. Shakevsky, A., Ronen, E., Wool, A.: Trust dies in darkness: Shedding light on samsung’s trustzone keymaster design. In: USENIX Security Symposium. pp. 251–268. USENIX Association (2022)
48. skylot: Jadx. <https://github.com/skylot/jadx> (2025)
49. Taylor, S.: 10 ways to tell if your android phone is hacked. <https://www.certosoftware.com/insights/how-to-tell-if-your-android-phone-is-hacked/> (2026)
50. Wallet, G.: Your secure digital wallet, backed by google. https://wallet.google/intl/en_us/ (2026)
51. WIRED: Inside the digital wallet revolution. <https://www.wired.com/sponsored/story/inside-the-digital-wallet-revolution-jp-morgan> (2022)
52. Zingg, S., Lain, D., Nakatsuka, Y., Kostiaainen, K., Bechtold, S., Čapkun, S.: Credential disclosure in (eu) digital identity wallets: Privacy risks and practical mitigations (2026), <https://arxiv.org/abs/2606.06354>
53. Zuo, C., Lin, Z., Zhang, Y.: Why does your data leak? uncovering the data leakage in cloud from mobile apps. In: IEEE Symposium on Security and Privacy. pp. 1296–1310. IEEE (2019)